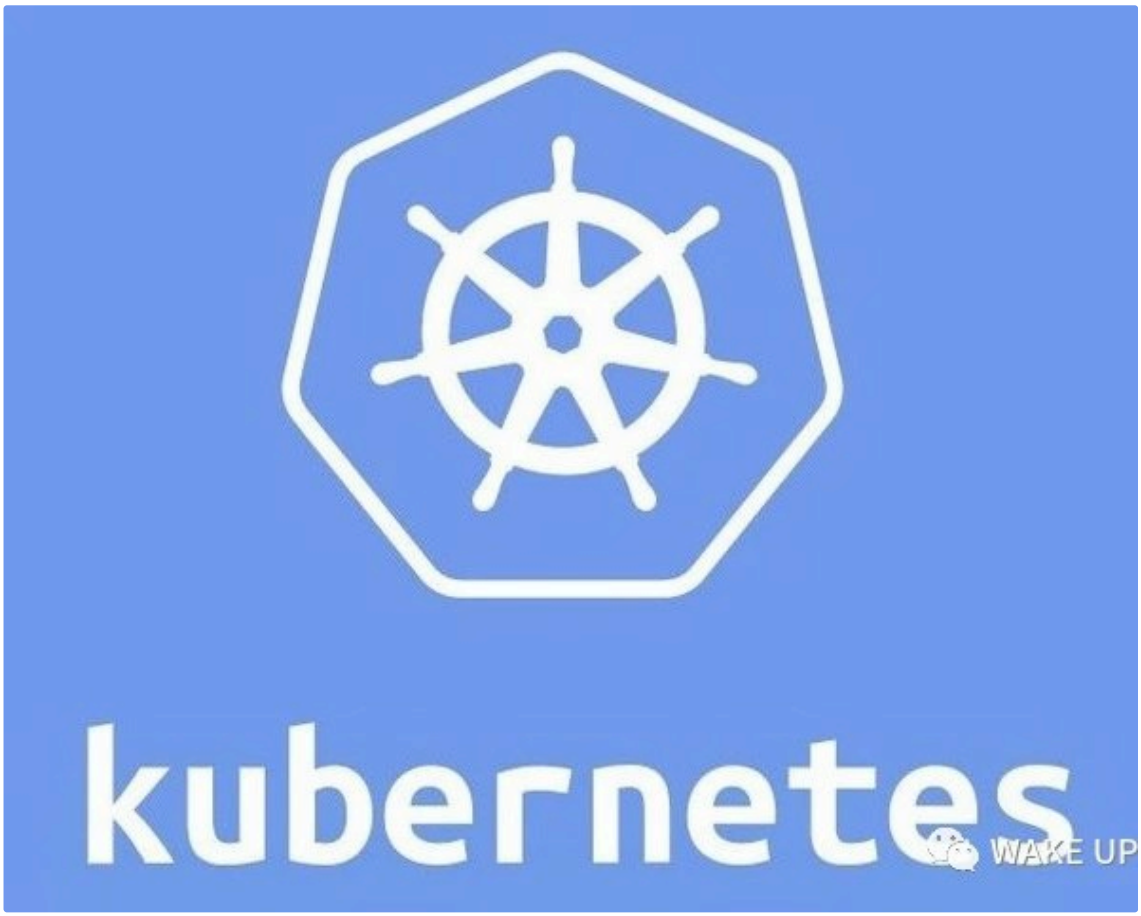
《K8S Nginx Ingress 详解》

Original WAKEUP技术 WAKE UP技术 2024年07月11日 23:28

给大家推荐一本趣味学习K8S的小说

起点读书

Kubernetes修仙



小程序

我是WAKE UP 专注于DevOps技术栈.

- 个人公众号: WAKE UP技术
- 个人主页: https://lweiqiang.xyz
- 个人博客: https://blog.lweiqiang.xyz
- 2024年加油！

《K8S Nginx Ingress 详解》

Nginx Ingress 详解

在 Kubernetes 环境中，Nginx Ingress 是一个强大的工具，用于实现服务发现和负载均衡。它为 Kubernetes 集群提供了从外部访问集群内服务的入口。 **一、什么是 Nginx Ingress**

Nginx Ingress 可以理解为基于 Nginx 的 Kubernetes 控制器，主要作用是将外部的请求转发到集群内不同的服务上。它相当于 Nginx、HAProxy 等负载均衡代理服务器。 当应用规模较小时，使用 NodePort 或许能满足需求，但随着应用增多，管理大量的端口会变得麻烦，此时 Nginx Ingress 的优势就体现出来了。

二、Nginx Ingress 的工作原理

Nginx Ingress 由两部分组成：Ingress controller（控制器）和 Ingress 服务。 Ingress controller 可以看作一个监听器，它通过不断监听 Kubernetes API Server，实时感知后端 Service、Pod 的变化。当获取到这些信息的变化后，结合 Ingress 的配置，更新反向代理负载均衡器，从而实现服务发现的作用。这点和服务发现工具 Consul、Consul-Template 非常类似。 具体工作原理如下：

1. Ingress controller 与 Kubernetes 的 API 进行交互，动态感知 Kubernetes 集群中 Ingress 服务规则的变化，并读取这些规则。
2. 根据规则中定义的域名与 Kubernetes 集群中 Service 的对应关系，再依据 Ingress controller 中的 Nginx 配置模板，生成相应的 Nginx 配置。
3. 将生成的配置动态写入到 Ingress controller 的 Pod 里（该 Pod 中运行着一个 Nginx 服务），然后重新加载配置使其生效，以此达到域名分配和动态更新的效果。

三、Nginx Ingress 的优势

1. **高度可扩展性和灵活性**：使用 Nginx 作为内部处理引擎，可轻松与许多不同的后端服务集成，并支持多种负载均衡算法，如轮询、最少连接和 IP 散列等。
2. **强大的功能集**：提供了诸多高级功能，例如 SSL 终止、健康检查、反向代理和流量控制等。
3. **易于集成**：能够轻松地与其他 Kubernetes 组件，如 Deployment、Service 和 ConfigMap 等进行集成。

四、Nginx Ingress 的安装与使用 以下是使用 Helm 安装 Nginx Ingress controller 的一般步骤（请注意，实际操作时需要根据你的环境和需求进行调整）：

1. 添加 Helm 仓库： `helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx`
2. 更新 Helm 仓库： `helm repo update`
3. 获取 ingress-nginx 包： `helm fetch ingress-nginx/ingress-nginx`
4. 解压包： `tar -xvf ingress-nginx-<版本>.tgz`
5. 更改资源清单文件（例如 values-prod.yaml），设置相关参数，如使用 hostnetwork 模式将 Ingress 固定到特定节点等。示例如下：

```
controller:
  name: controller
```

```
    image:
      repository: cnych/ingress-nginx
      tag: "v0.41.2"
      digest:
    dnsPolicy: clusterFirstWithHostNet
    hostnetwork: true
    publishService: # hostnetwork 模式下设置为 false，通过节点 ip 地址上报 ingress s
      enabled: false
    kind: daemonset
    nodeSelector:
      role: lb
    service: # hostnetwork 模式不需要创建 service
      enabled: false
  defaultBackend:
    enabled: true
    name: defaultbackend
    image:
      repository: cnych/ingress-nginx-defaultbackend
      tag: "1.5"
```

1. 使用以下命令安装到指定命名空间： `kubectl create namespace ingress-nginx  helm install --namespace ingress-nginx ingress-nginx./ingress-nginx -f./ingress-nginx/values-prod.yaml`

安装完成后，就可以定义 Ingress 资源来描述服务的暴露规则，例如域名、路径和后端服务等，并将其应用到 Kubernetes 集群中。

五、Ingress 资源的关键部分 Ingress 资源的一些重要字段和注解包括：

- `metadata.name`： Ingress 资源的名称。
- `spec.rules.host`： 定义匹配的域名。
- `spec.rules.http.paths.path`： 具体的路径规则。
- `spec.rules.http.paths.backend.serviceName`： 后端服务的名称。
- `spec.rules.http.paths.backend.servicePort`： 后端服务的端口。
- 注解（Annotations）：例如 `kubernetes.io/ingress.class: "nginx"` 用于指定使用 Nginx Ingress controller；其他注解还可用于配置 SSL 终止、超时时间、代理设置等各种功能。

例如，以下是一个简单的 Ingress 资源定义示例：

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
```

```
    kubernetes.io/ingress.class: "nginx"
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "30"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "1800"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "1800"
    nginx.ingress.kubernetes.io/proxy-body-size: "8m"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  tls:
    - hosts:
      - 'https://example.com'
      secretName: tls-cert
  rules:
    - host: 'https://example.com'
      http:
        paths:
          - path: /
            backend:
              serviceName: my-service
              servicePort: 80
```

在上述示例中，定义了一个名为 `my-ingress` 的 Ingress 资源，使用 Nginx Ingress controller，配置了 SSL 证书相关信息，并将 `https://example.com/` 的请求转发到名为 `my-service` 的服务的 80 端口。

六、常见应用场景

1. **多服务暴露**：当集群中有多个服务需要对外暴露时，通过不同的域名或路径规则，将请求分发到相应的服务。
2. **路径分流**：根据请求的路径，将流量导向到不同的后端服务。
3. **SSL 终止**：在 Ingress 层处理 SSL/TLS 加密和解密，后端服务可以直接接收明文请求。
4. **流量控制**：例如限制某些路径的访问频率或并发连接数等。

通过合理配置 Nginx Ingress，可以实现灵活、高效的服务暴露和管理，提高 Kubernetes 应用的可访问性和可用性。同时，结合监控和日志系统，可以更好地跟踪和分析服务的运行状况。在实际使用中，还需要根据具体的业务需求和网络环境，进行细致的配置和优化。

实战优化

Nginx Ingress 是 Kubernetes 中用于管理外部流量访问集群内部服务的重要组件。

Ingress 出现是为应对 Kubernetes 中 Pod 挂掉时 IP 变化导致的服务发现问题以及配置基于 HTTP 负载均衡的需求。它将 Kubernetes 内部的服务暴露出去，提供基于 HTTP 层的流量负载

均衡（基于主机头或者 URL），还提供 TLS/SSL 加密。其核心概念包括：

- Host：未指定 Host 时，规则适用于通过指定 IP 地址的所有入站 HTTP 通信。若提供了 Host（如 foo.bar.com），则 rules 适用于该 Host。
- Rules：用于定义当前 Ingress 资源的转发规则列表。
- Backend：默认的后端用于服务那些没有匹配到任何规则的请求。

Nginx Ingress 的工作原理为：Ingress-controller 通过和 API server 交互，动态获取 Ingress 的规则变化。当 Ingress 发生变化时，按照固定格式生成 Nginx 可以识别的前后端配置文件，再将这段配置文件写入到 Ingress-controller 中的 Nginx 服务中，然后 reload 一下使配置文件生效。

在功能方面，Ingress 是通过 Service 的服务发现功能来发现后端的 Pod，然后流量直接发给 Pod，而不经由 Service，因此比 NodePort 的方式网络损耗更小。

Kubernetes 中的 Service 和 Pod 的 IP 可能变化，而 Ingress 及 Nginx Ingress 控制器能够有效管理外部流量的路由和负载均衡，为集群内服务的暴露和访问提供了灵活且高效的方式。

Nginx ingress 的配置方法

Nginx Ingress 是 Kubernetes 中用于管理外部流量访问集群内部服务的重要组件。配置 Nginx Ingress 通常包括以下几个方面： 首先，将 Nginx 的配置抽象成一个 Ingress 对象，每添加一个新的服务只需写一个新的 Ingress 的 yaml 文件即可。Ingress 由 ingress controller、ingress 服务等组成。 ingress controller 通过与 k8s 的 api 进行交互，动态地去感知 k8s 集群中 ingress 服务规则的变化，然后读取它，并按照定义的 ingress 规则，转发到 k8s 集群中对应的 service。 例如，部署 httpd 服务时，需要创建相应的 pod 和 service，并通过配置 Ingress 规则来实现外部访问。 Ingress 能够解决动态配置服务的问题。当新增加一个服务时，只需配置好这个服务，而无需在流量入口加一个反向代理指向新的 k8s 服务。

Nginx ingress 与其他 ingress 控制器的比较

Istio-Ingress 和 Nginx-Ingress 有以下主要差异： 架构方面，Istio-Ingress 是 Istio Service Mesh 的一部分，由 Envoy 代理实现，与 Istio 的其他组件协作；Nginx-Ingress 是一个独立的控制器，由 Nginx 实现，不依赖其他 Service Mesh 组件。 功能上，Istio-Ingress 提供了更丰富的流量管理功能，如版本路由、熔断、重试等；Nginx-Ingress 主要提供基本的 HTTP/HTTPS 路由和负载均衡功能。 集成方面， Istio-Ingress 能与 Istio 的其他组件无缝集成，提供统一的配置和管理体验；Nginx-Ingress 作为独立组件，需要与 Istio 的其他组件进行集成和配合。 复杂度上， Istio-Ingress 作为 Istio 的一部分，相对更复杂，需要部署和配置整个 Istio 系统；Nginx-Ingress 部署和配置相对简单，只需部署控制器即可。

APISIX Ingress 与 Nginx-Ingress 也有不同。APISIX Ingress 内置的功能和特性相比 Nginx-Ingress 更加丰富，包括服务发现、协议支持、认证鉴权等。

总的来说，选择哪种 ingress 控制器取决于具体的应用场景和需求。

Nginx ingress 如何优化性能

优化 Nginx Ingress 的性能可以从多个方面入手： 调整连接队列大小，将 nginx 进程监听 socket 套接字的连接队列大小从默认的 511 增大，以应对高并发场景下的业务流量洪峰。 调整默认端口使用范围，在高并发场景下，默认的端口资源可能不足，通过修改端口范围可以避免无法与 upstream 服务端建立连接的问题。 增大最大文件描述符数限制，以同时处理海量请求。 处理 TIME_WAIT 连接，启用 net.ipv4.tcp_tw_reuse 选项，复用 TIME_WAIT 状态的 socket 用于新建连接，避免端口资源被长时间占用。 还可以优化 ingress-controller，例如通过修改资源对象 Deployment/DaemonSet 实例中的参数来自定义关联配置映射实例的名称。 同时，参考 ingress 的 Annotations 配置方法，优化具体 deployment.yml 里的 ingress 配置，设置合理的超时时间等参数。

Nginx ingress 常见错误及解决办法

常见的错误包括： 当请求头的值命中指定的自定义值时，请求可能无法正确转发给 Canary Ingress 定义的对应后端服务。 流量高负载情况下健康检查失败，可能需要检查证书配置、SLB 监听设置等。 Nginx Ingress 证书不匹配，可能是由于证书的版本、域名等不匹配导致，需要仔细检查证书的相关配置。 Nginx 大压力流量下占用大量内存，可能需要调整内存相关的配置参数。 对于 cert-manager 不符合预期导致证书签发失败的情况，需要检查 cert-manager 的配置和运行状态。

针对这些错误，需要根据具体情况进行排查和解决。例如，检查域名上是否启用了 WAF 接入或透明接入，检查 SLB 是否为七层监听等。

Nginx ingress 实际应用案例

在实际应用中，Nginx Ingress 有很多场景。例如，部署后端 Jenkins 的 pod 和 service，通过创建相应的 yaml 文件来定义服务和部署。 在实现灰度发布方面，Nginx Ingress 支持基于 Header、Cookie 和服务权重等多种流量切分策略。假设线上已运行了一套对外提供服务的 Service A，开发了新特性需要发布上线新版本 Service A'，可以通过配置 Nginx Ingress，将带有特定 Header 或 Cookie 的用户请求转发到新版本服务，实现灰度发布。

综上所述，Nginx Ingress 在 Kubernetes 中发挥着重要作用，通过合理的配置、优化和解决常见问题，可以更好地满足服务暴露和流量管理的需求，并在实际应用中根据不同场景灵活运用其功能。

推荐阅读:

起点读书

Kubernetes修仙



小程序

如果你喜欢我们的文章，欢迎点赞、打赏、转发支持我们。最重要的还是要谢谢大家对本公众号的支持，我们将一如既往地推送技术干货、及时推送最新技术.....

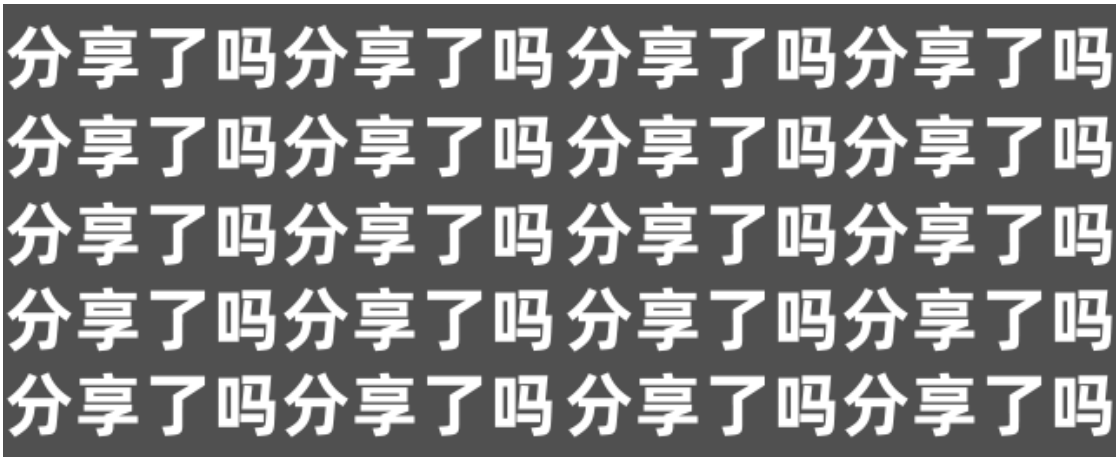


WAKE UP技术
DevOps 技术分享
155篇原创内容

公众号

有技术问题欢迎扫描下方二维码加我微信讨论.





负载均衡 6 kubernetes 49 nginx 11 linux 84 云原生 78

负载均衡 · 目录

上一篇
K8S 之Ingress 详解

下一篇
K8S 运维踩坑血泪史：那些痛与成长